

Integrating single-cell transcriptomes from multiple samples

Install the required libraries

```
In [ ]: download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_cranapt_jammy.s
          "add_cranapt_jammy.sh")
Sys.chmod("add_cranapt_jammy.sh", "0755")
system("./add_cranapt_jammy.sh")
bspm::enable()
options(bspm.version.check=FALSE)
```

We will create an R function to performs system calls

```
In [ ]: shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...)
  cat(paste0(result, collapse = "\n"))
}
```

Install required libraries

```
In [ ]: # Install the R.utils package
install.packages("R.utils")

# Install a specific commit/version of the Seurat Wrappers package from GitHub
remotes::install_github('satijalab/seurat-wrappers@d28512f804d5fe05e6d68900ca9221020d52cf1d',

# Install the Seurat Data package from GitHub
remotes::install_github('satijalab/seurat-data')

# Check if the BiocManager package is installed; if not, install it quietly
if (!require("BiocManager", quietly = TRUE))
  install.packages("BiocManager", quiet = TRUE)

# Install the Harmony package for integrating single-cell datasets
install.packages("harmony")
```

```
In [23]: remotes::install_github("immunogenomics/harmony")
```

Using github PAT from envvar GITHUB_PAT. Use `gitcreds::gitcreds\_set()` and unset GITHUB_PAT in .Renviron (or elsewhere) if you want to use the more secure git credential store instead.

Downloading GitHub repo immunogenomics/harmony@HEAD

```
* checking for file '/tmp/RtmpezIKtu/remotesb1745caaf0f/immunogenomics-harmony-200114c/DESCRIPTION' ... OK
* preparing 'harmony':
* checking DESCRIPTION meta-information ... OK
* cleaning src
* checking for LF line-endings in source and make files and shell scripts
* checking for empty or unneeded directories
* building 'harmony_1.2.4.tar.gz'
```

Introduction

Integration is a technique used to combine single cell data from different experiments or conditions into a single analysis space. This is especially useful where data from different batches or conditions can be integrated to identify common biological patterns. In this example of integrating multiple datasets representing, for example, different biological conditions, sample handling, or experimental protocols, we will explore the integration performed by [Seurat](#) and [Harmony](#) separately.

```
In [ ]: # Load Libraries
library(Seurat)
library(SeuratData)
library(SeuratWrappers)
library(cowplot)
library(harmony)
# If the time is over than 2000 seconds, the work will break
options(timeout=2000)
```

Loading Data

Here, we load the datasets necessary to perform an integration analysis.

This tutorial demonstrates the integration process using peripheral blood mononuclear cells (PBMCs) data from the study by [Kang et al, 2017](#). The experiment involved splitting PBMCs into two groups: one control group and one stimulated group treated with interferon beta. The treatment induced cell type-specific gene expression changes, creating a challenge for joint analysis. Without integration, cells may cluster based on their stimulation condition rather than their intrinsic cell type identity, complicating the interpretation of results.

To clarify, the integration aims to align data from both groups into a common space where cells cluster by type rather than treatment condition. This process is essential for accurately identifying shared and unique features across the datasets while minimizing the influence of batch effects introduced by the experimental design.

Steps to Follow:

1. Load the PBMC datasets: Use preprocessed gene expression data from the Kang study.
2. Inspect the dataset composition: Verify that both control and stimulated groups are included and check for adequate cell type representation.

3. Perform integration: Employ integration tools such as Seurat or Harmony to combine the datasets while accounting for batch effects.
4. Visualize the results: Examine clustering to ensure cells group primarily by type rather than by treatment condition. These steps ensure a robust and biologically meaningful comparison of the control and stimulated PBMC groups.

```
In [ ]: # install dataset  
InstallData("ifnb")
```

```
In [2]: # Load dataset  
LoadData("ifnb")
```

Validating object structure

Updating object slots

Ensuring keys are in the proper structure

Warning message:

“Assay RNA changing from Assay to Assay”

Ensuring keys are in the proper structure

Ensuring feature names don't have underscores or pipes

Updating slots in RNA

Validating object structure for Assay ‘RNA’

Object representation is consistent with the most current Seurat version

Warning message:

“Assay RNA changing from Assay to Assay5”

An object of class Seurat

14053 features across 13999 samples within 1 assay

Active assay: RNA (14053 features, 0 variable features)

2 layers present: counts, data

```
In [3]: # Transfer ifnb data to sc_dataset  
sc_datasets <- ifnb
```

```
In [6]: # %>%: This is the operator from the dplyr package. It allows you to chain operations in a read  
# glimpse is a function of the dplyr package that provides a quick and concise view of the data  
  
sc_datasets %>% dplyr::glimpse()
```

```

Formal class 'Seurat' [package "Seurat"] with 13 slots
..@ assays          :List of 1
.. ..$ RNA:Formal class 'Assay' [package "Seurat"] with 8 slots
..@ meta.data       :'data.frame':      13999 obs. of  5 variables:
.. ..$ orig.ident    : chr [1:13999] "IMMUNE_CTRL" "IMMUNE_CTRL" "IMMUNE_CTRL" "IMMUNE_CT
RL" ...
.. ..$ nCount_RNA     : num [1:13999] 3017 2481 3420 3156 1868 ...
.. ..$ nFeature_RNA   : int [1:13999] 877 713 850 1109 634 557 980 581 880 669 ...
.. ..$ stim           : chr [1:13999] "CTRL" "CTRL" "CTRL" "CTRL" ...
.. ..$ seurat_annotati: Factor w/ 13 levels "CD14 Mono","CD4 Naive T",...: 1 1 1 12 3 1 7 2
6 1 ...
..@ active.assay: chr "RNA"
..@ active.ident: Factor w/ 2 levels "IMMUNE_CTRL",...: 1 1 1 1 1 1 1 1 1 1 ...
.. ..- attr(*, "names")= chr [1:13999] "AAACATACATTTC.1" "AAACATACCAGAAA.1" "AAACATACCTCGCT.
1" "AAACATACCTGGTA.1" ...
..@ graphs          : list()
..@ neighbors        : list()
..@ reductions       : list()
..@ project.name: chr "ifnb"
..@ misc             : list()
..@ version          :Classes 'package_version', 'numeric_version'  hidden list of 1
.. ..$ : int [1:3] 3 0 0
..@ commands         : list()
..@ tools            : list()
..@ NA               : NULL

```

Data Preprocessing

We preprocess the data to a point that is in common for both Seurat and Harmony data integration,

```

In [7]: # Update the Seurat object to the latest format, fixing compatibility issues with older Seurat
sc_datasets <- UpdateSeuratObject(object = sc_datasets)
# If this step is skipped, an error occurs: 'no slot of name "images" for this object of class

# Normalize the gene expression data in the Seurat object
sc_datasets <- Seurat::NormalizeData(sc_datasets, verbose = FALSE)

```

Validating object structure

Updating object slots

Ensuring keys are in the proper structure

Ensuring keys are in the proper structure

Ensuring feature names don't have underscores or pipes

Updating slots in RNA

Validating object structure for Assay 'RNA'

Object representation is consistent with the most current Seurat version

The `NormalizeData` function in Seurat simplifies raw single-cell RNA sequencing (scRNA-seq) data into a standardized form, making it easier to compare cells and detect meaningful biological patterns. This process involves three main steps:

1. **Scaling Counts:** For each gene, the count in a given cell is divided by the total number of counts from all genes in that cell. This adjusts for differences in sequencing depth across cells, ensuring fair comparisons.
2. **Adjusting Scale:** The result from the first step is multiplied by a scaling factor (commonly set to 10,000). This makes the normalized values more interpretable, roughly resembling counts per 10,000 reads.
3. **Log Transformation:** Finally, the scaled counts are transformed using a logarithmic function. This step reduces the effect of extremely high or low values, making patterns in the data easier to detect.

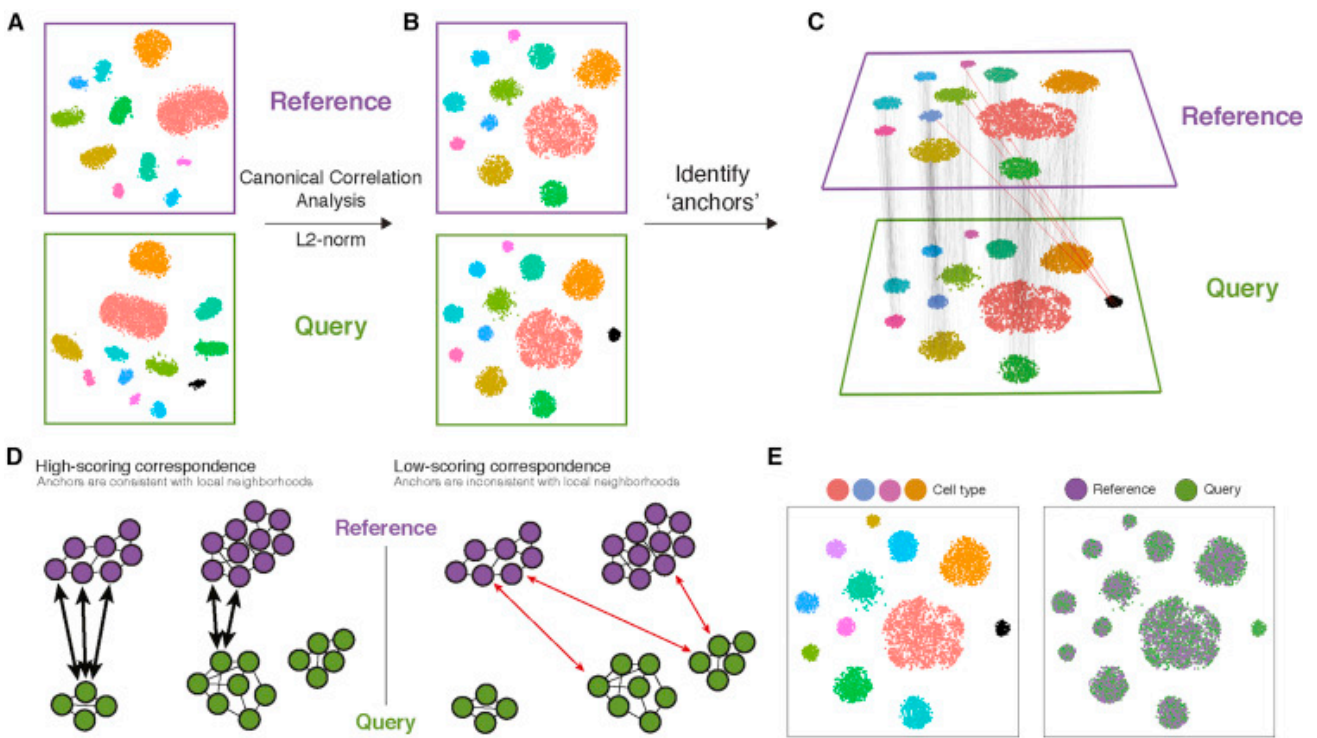
This method is essential because it ensures that any differences observed between cells reflect true biological variations, not technical differences from the sequencing process

```
In [8]: # View the structure of the Seurat object, including metadata and feature data
dplyr::glimpse(sc_datasets)
```

```
Formal class 'Seurat' [package "SeuratObject"] with 13 slots
  ..@ assays      :List of 1
  .. ..$ RNA:Formal class 'Assay' [package "SeuratObject"] with 8 slots
  ..@ meta.data   :'data.frame':      13999 obs. of  5 variables:
  .. ..$ orig.ident      : chr [1:13999] "IMMUNE_CTRL" "IMMUNE_CTRL" "IMMUNE_CTRL" "IMMUNE_CT
RL" ...
  .. ..$ nCount_RNA      : num [1:13999] 3017 2481 3420 3156 1868 ...
  .. ..$ nFeature_RNA    : int [1:13999] 877 713 850 1109 634 557 980 581 880 669 ...
  .. ..$ stim            : chr [1:13999] "CTRL" "CTRL" "CTRL" "CTRL" ...
  .. ..$ seurat_annotat: Factor w/ 13 levels "CD14 Mono","CD4 Naive T",...: 1 1 1 12 3 1 7 2
6 1 ...
  ..@ active.assay: chr "RNA"
  ..@ active.ident: Factor w/ 2 levels "IMMUNE_CTRL",...: 1 1 1 1 1 1 1 1 1 ...
  .. ..- attr(*, "names")= chr [1:13999] "AAACATACATTTCC.1" "AAACATACCAGAAA.1" "AAACATACCTCGCT.
1" "AAACATACCTGGTA.1" ...
  ..@ graphs      : list()
  ..@ neighbors    : list()
  ..@ reductions   : list()
  ..@ images       : list()
  ..@ project.name: chr "ifnb"
  ..@ misc         : list()
  ..@ version      :Classes 'package_version', 'numeric_version'  hidden list of 1
  .. ..$ : int [1:3] 5 3 0
  ..@ commands     :List of 1
  .. ..$ NormalizeData.RNA:Formal class 'SeuratCommand' [package "SeuratObject"] with 5 slots
  ..@ tools        : list()
```

Run Seurat data integration

The integration performed here with Seurat consists of using a Canonical Correlation Analysis to identify anchors between datasets, as indicated in the [original paper](#)(30559-8):



Split dataset into a list of datasets

```
In [9]: # Split the Seurat object into a list of smaller Seurat objects based on the "stim" metadata c
sc_datasets.list <- SplitObject(sc_datasets, split.by = "stim")
```

Find highly variable features for each condition separately

```
In [10]: # Identify the top 2000 variable features for each dataset in the list
sc_datasets.list <- lapply(X = sc_datasets.list, FUN = function(x) {
  x <- FindVariableFeatures(x, selection.method = "vst", nfeatures = 2000)
})
```

Find the features that are repeatedly variable across datasets for integration (anchor features).

```
In [11]: # Identify integration features to be used for aligning datasets
features <- SelectIntegrationFeatures(object.list = sc_datasets.list)
```

Find the integration anchors or final set of highly variable genes most frequently selected across the batches

```
In [12]: # Find integration anchors between the datasets using the selected features
sc_datasets.anchors <- FindIntegrationAnchors(object.list = sc_datasets.list, anchor.features

# Below is the resume of the running process that will appear in your screen
```

Scaling features for provided objects

Finding all pairwise anchors

Running CCA

Merging objects

Finding neighborhoods

Finding anchors

Found 16393 anchors

Filtering anchors

Retained 6756 anchors

Integrate datasets - Creates an integrated data assay

```
In [13]: # Integrate the datasets into a single Seurat object using the integration anchors  
sc_datasets.combined <- IntegrateData(anchorset = sc_datasets.anchors)  
  
# Below is the resume of the running process that will appear in your screen
```

Merging dataset 1 into 2

Extracting anchors for merged samples

Finding integration vectors

Finding integration vector weights

Integrating data

```
In [14]: # View the structure of the integrated Seurat object  
dplyr::glimpse(sc_datasets.combined)
```

```

Formal class 'Seurat' [package "SeuratObject"] with 13 slots
..@ assays          :List of 2
.. ..$ RNA          :Formal class 'Assay' [package "SeuratObject"] with 8 slots
.. ..$ integrated:Formal class 'Assay' [package "SeuratObject"] with 8 slots
..@ meta.data       :'data.frame':      13999 obs. of  5 variables:
.. ..$ orig.ident    : chr [1:13999] "IMMUNE_CTRL" "IMMUNE_CTRL" "IMMUNE_CTRL" "IMMUNE_CT
RL" ...
.. ..$ nCount_RNA     : num [1:13999] 3017 2481 3420 3156 1868 ...
.. ..$ nFeature_RNA   : int [1:13999] 877 713 850 1109 634 557 980 581 880 669 ...
.. ..$ stim           : chr [1:13999] "CTRL" "CTRL" "CTRL" "CTRL" ...
.. ..$ seurat_annotat: chr [1:13999] "CD14 Mono" "CD14 Mono" "CD14 Mono" "pDC" ...
..@ active.assay: chr "integrated"
..@ active.ident: Factor w/ 2 levels "IMMUNE_CTRL",...: 1 1 1 1 1 1 1 1 1 1 ...
.. - attr(*, "names")= chr [1:13999] "AAACATACATTTCC.1" "AAACATACCAGAAA.1" "AAACATACCTCGCT.
1" "AAACATACCTGGTA.1" ...
..@ graphs          : list()
..@ neighbors       : list()
..@ reductions      : list()
..@ images          : list()
..@ project.name: chr "SeuratProject"
..@ misc            : list()
..@ version         :Classes 'package_version', 'numeric_version' hidden list of 1
.. ..$ : int [1:3] 5 3 0
..@ commands        :List of 2
.. ..$ FindIntegrationAnchors:Formal class 'SeuratCommand' [package "SeuratObject"] with 5 sl
ots
.. ..$ withCallingHandlers :Formal class 'SeuratCommand' [package "SeuratObject"] with 5 sl
ots
..@ tools           :List of 1
.. ..$ Integration:Formal class 'IntegrationData' [package "Seurat"] with 7 slots

```

Run the standard workflow for visualization and clustering.

```

In [ ]: # Scale the integrated data, run PCA, perform UMAP for dimensionality reduction, find neighbors
sc_datasets.combined <- ScaleData(sc_datasets.combined, verbose = FALSE) %>%
  RunPCA(npcs = 30, verbose = FALSE) %>%
  RunUMAP(reduction = "pca", dims = 1:30) %>%
  FindNeighbors(reduction = "pca", dims = 1:30) %>%
  FindClusters(resolution = 0.5)

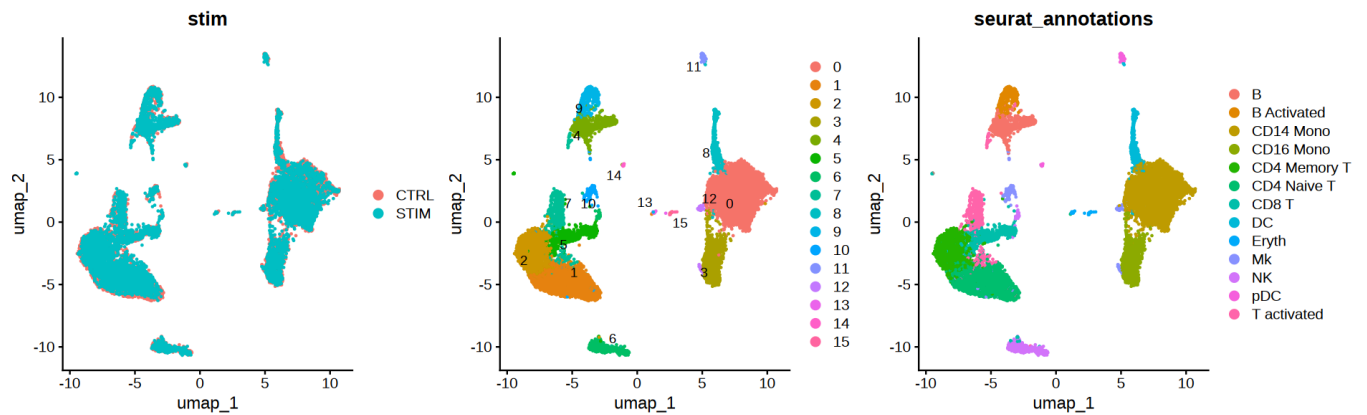
```

```

In [16]: # Set the size of the plots for visualization
options(repr.plot.height = 5, repr.plot.width = 16)

# Visualization: Create UMAP plots with different grouping or labeling options
p1 <- DimPlot(sc_datasets.combined, reduction = "umap", group.by = "stim") # Group cells by "stim"
p2 <- DimPlot(sc_datasets.combined, reduction = "umap", label = TRUE, repel = TRUE) # Label cells
p3 <- DimPlot(sc_datasets.combined, reduction = "umap", group.by = "seurat_annotat") # Group cells by "seurat_annotat"
p1 + p2 + p3 # Combine the plots

```

Run Harmony

From the [original paper](#) of harmony: "PCA embeds cells into a space with reduced dimensionality.

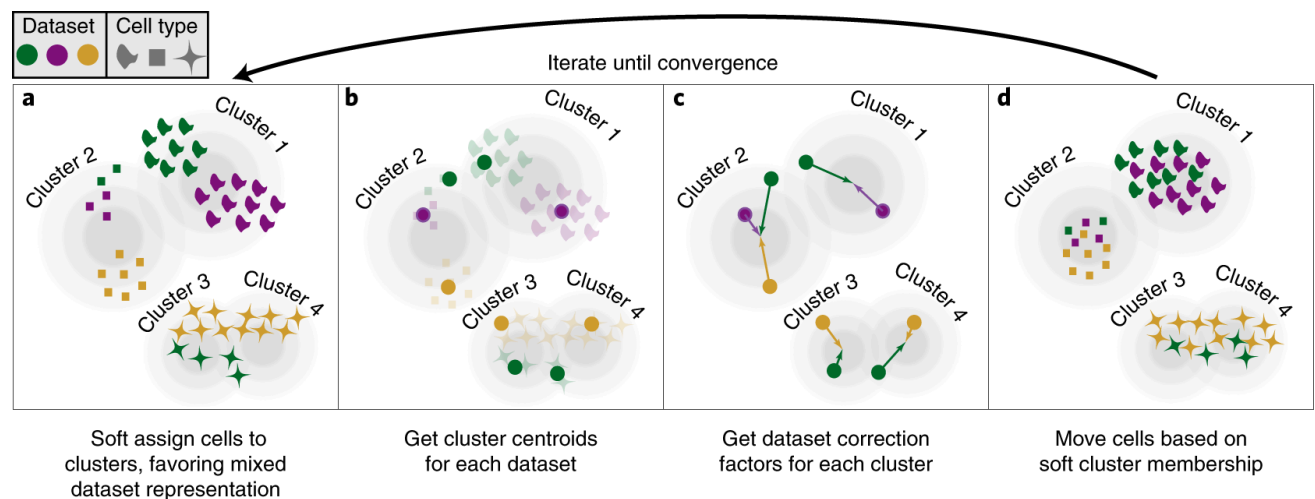
Harmony accepts the cell coordinates in this reduced space and runs an iterative algorithm to adjust for dataset specific effects.

a. Harmony uses fuzzy clustering to assign each cell to multiple clusters, while a penalty term ensures that the diversity of datasets within each cluster is maximized.

b. Harmony calculates a global centroid for each cluster, as well as dataset-specific centroids for each cluster.

c. Within each cluster, Harmony calculates a correction factor for each dataset based on the centroids.

d. Finally, Harmony corrects each cell with a cell-specific factor: a linear combination of dataset correction factors weighted by the cell's soft cluster assignments made in step a. Harmony repeats steps a to d until convergence. The dependence between cluster assignment and dataset diminishes with each round. Datasets are represented with colors, cell types with different shapes."



```
In [17]: # Set the plot dimensions for visualization
options(repr.plot.height = 4, repr.plot.width = 6)

# Find variable features, scale data, run PCA, and integrate using Harmony
```

```
sc_datasets.harmony <- FindVariableFeatures(sc_datasets, selection.method = "vst", nfeatures = 3000)
sc_datasets.harmony <- ScaleData(verbose = FALSE) %>%
  RunPCA(npcs = 30, verbose = FALSE) %>%
  RunHarmony("stim", plot_convergence = TRUE) # Integrate using "stim" and "vst"
```

Transposing data matrix

Initializing state using k-means centroids initialization

Harmony 1/10

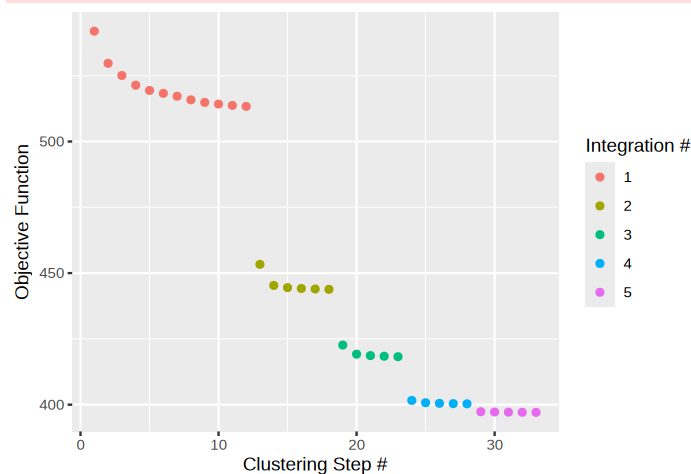
Harmony 2/10

Harmony 3/10

Harmony 4/10

Harmony 5/10

Harmony converged after 5 iterations



To directly access the harmonized embeddings:

```
In [18]: # Extract Harmony embeddings (low-dimensional representations) and display the first 5 rows and columns
harmony_embeddings <- Embeddings(sc_datasets.harmony, 'harmony')
harmony_embeddings[1:5, 1:5]
```

A matrix: 5 × 5 of type dbl

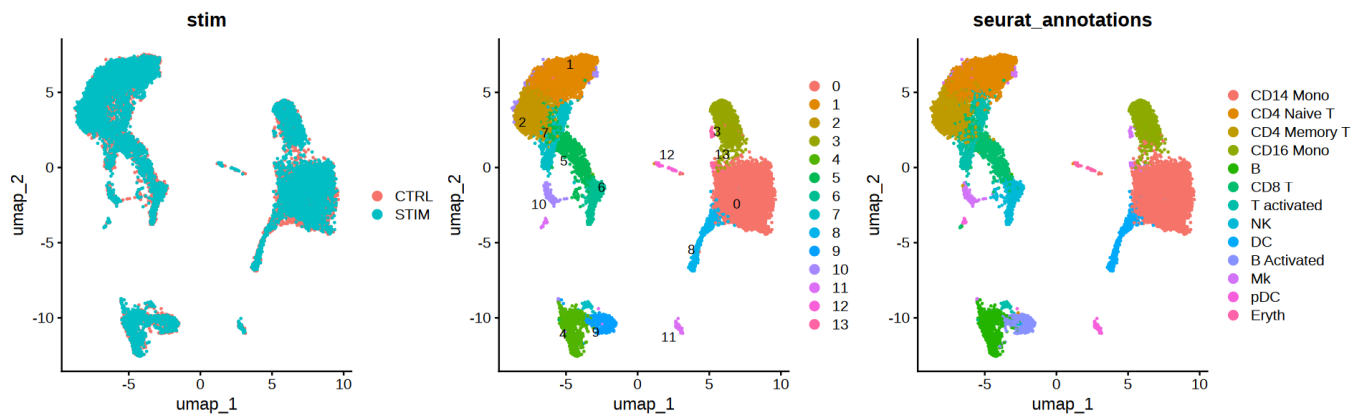
	harmony_1	harmony_2	harmony_3	harmony_4	harmony_5
AAACATACATTTC.1	-11.516710	0.9760470	1.8251243	0.05220736	0.3106200
AAACATACCAGAAA.1	-12.022023	2.5844361	-2.7820106	0.42781200	-1.6178475
AAACATACCTCGCT.1	-9.647215	2.6329526	-0.3382079	0.85529087	0.4971925
AAACATACCTGGTA.1	0.896209	-1.9783555	13.3801892	-5.94854557	-1.2596881
AAACATACGATGAA.1	7.122691	0.1068368	-1.4049431	2.59020042	-0.1806337

Complete the standard workflow for visualization and clustering

```
In [ ]: # Run UMAP using Harmony embeddings, find neighbors, and cluster cells
sc_datasets.harmony <- RunUMAP(sc_datasets.harmony, reduction = "harmony", dims = 1:30) %>%
  FindNeighbors(reduction = "harmony", dims = 1:30) %>%
  FindClusters(resolution = 0.5)
```

```
In [20]: # Set plot dimensions for visualization
options(repr.plot.height = 5, repr.plot.width = 16)

# Visualization: Create UMAP plots with different grouping or labeling options
p1 <- DimPlot(sc_datasets.harmony, reduction = "umap", group.by = "stim") # Group by "stim"
p2 <- DimPlot(sc_datasets.harmony, reduction = "umap", label = TRUE, repel = TRUE) # Label cl
p3 <- DimPlot(sc_datasets.harmony, reduction = "umap", group.by = "seurat_annotatations") # Gro
p1 + p2 + p3 # Combine the plots
```



Integration Metrics

Inspect mixing within cluster

We will use a function to plot the batch composition of each cluster.

This function was borrowed from:

https://github.com/cellgeni/scRNA.seq.course/blob/master/course_files/utils/custom_seurat_functions.R

```
In [21]: # Define a custom function to visualize the distribution of clusters across datasets
plot_integrated_clusters = function(srat, batchcolumn) {
  ## Take an integrated Seurat object and plot distributions across the "batchcolumn"
  library(Seurat)
  library(patchwork)
  library(ggplot2)
  library(reshape2)
  library(RColorBrewer)

  # Create a table of cell counts by cluster and dataset
  count_table <- table(srat@meta.data$seurat_clusters, srat@meta.data[[batchcolumn]])
  count_mtx <- as.data.frame.matrix(count_table) # Convert table to a data frame
  count_mtx$cluster <- rownames(count_mtx) # Add cluster identifiers as a column
  melt_mtx <- melt(count_mtx) # Reshape data for ggplot
  melt_mtx$cluster <- as.factor(melt_mtx$cluster) # Convert cluster column to a factor

  # Calculate cluster sizes
```

```

cluster_size <- aggregate(value ~ cluster, data = melt_mtx, FUN = sum)

# Sort clusters by size
sorted_labels <- paste(sort(as.integer(levels(cluster_size$cluster)), decreasing = TRUE))
cluster_size$cluster <- factor(cluster_size$cluster, levels = sorted_labels)
melt_mtx$cluster <- factor(melt_mtx$cluster, levels = sorted_labels)

colnames(melt_mtx)[2] <- "dataset" # Rename column for clarity

# Plot the distribution of cells per cluster (log scale)
p1 <- ggplot(cluster_size, aes(y = cluster, x = value)) +
  geom_bar(position = "dodge", stat = "identity", fill = "grey60") +
  theme_bw() + scale_x_log10() + xlab("Cells per cluster, log10 scale") + ylab("")

# Plot the fraction of cells in each dataset for each cluster
p2 <- ggplot(melt_mtx, aes(x = cluster, y = value, fill = dataset)) +
  geom_bar(position = "fill", stat = "identity") +
  theme_bw() + coord_flip() +
  scale_fill_brewer(palette = "Set2") +
  ylab("Fraction of cells in each dataset") + xlab("Cluster number") + theme(legend.posi

p2 + p1 + plot_layout(widths = c(3, 1)) # Combine plots
}

```

```

In [22]: # Visualize clusters for integrated datasets
plot_integrated_clusters(sc_datasets.combined, 'stim')

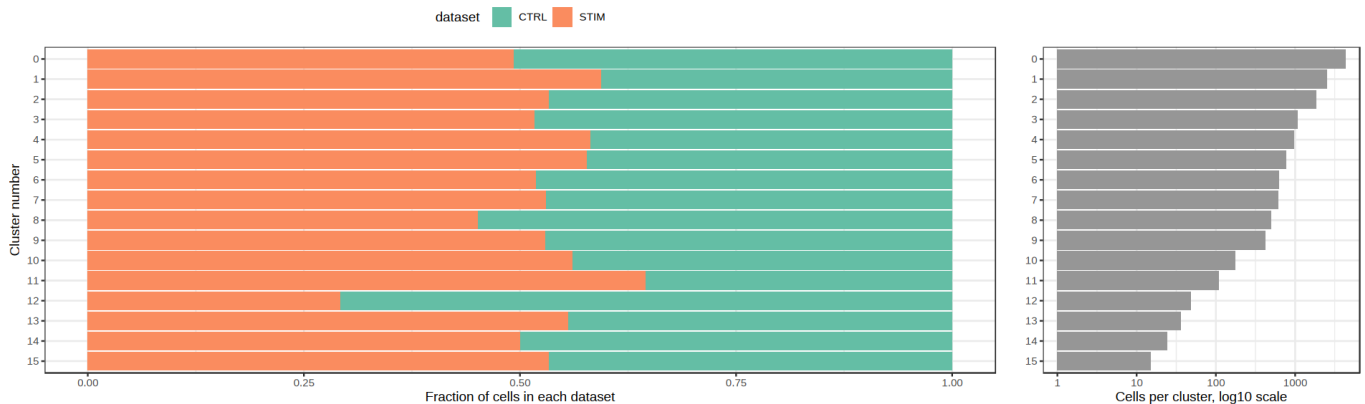
```

Attaching package: ‘patchwork’

The following object is masked from ‘package:cowplot’:

align_plots

Using cluster as id variables

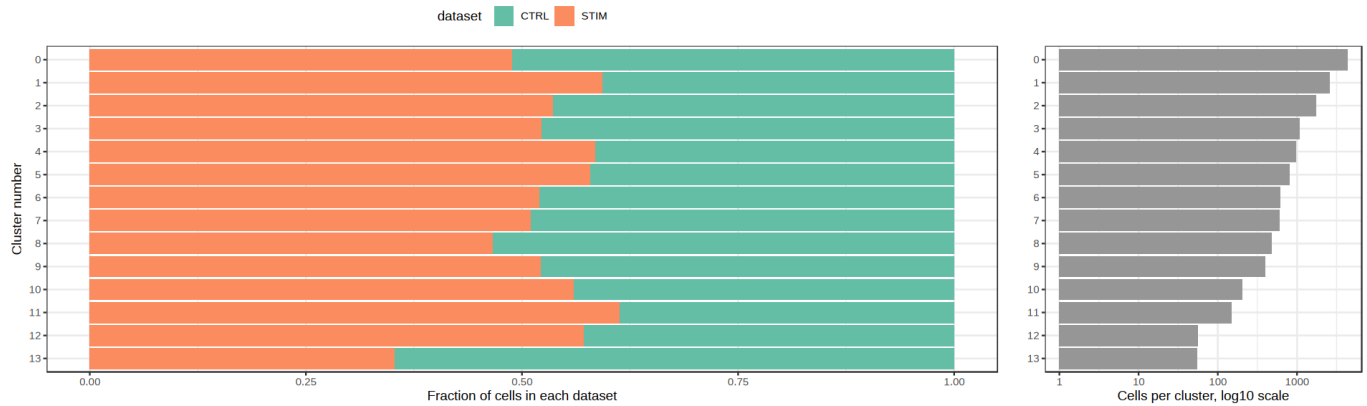


```

In [23]: # Visualize clusters for harmony
plot_integrated_clusters(sc_datasets.harmony, 'stim')

```

Using cluster as id variables



This measures how well mixed a composite dataset is. According to [this link](#), lower scores represent better mixing. See also [this code](#)

This metric evaluates whether the neighborhood of a cell is well mixed. In other words, whether it contains a small number of cells from each dataset (e.g., $k=5$).

```
In [24]: # Calculate the mixing metric for Seurat integrated data
seurat_mixing <- MixingMetric(sc_datasets.combined,
                              'seurat_clusters', # Column to evaluate mixing
                              reduction = "pca", # Use PCA embeddings
                              dims = 1:2,        # Evaluate the first two dimensions
                              k = 5,             # Number of nearest neighbors
                              max.k = 300,       # Maximum number of neighbors
                              eps = 0,           # Tolerance for neighbor search
                              verbose = TRUE     # Print progress
                              )
```

```
In [25]: # Calculate the mean of the Seurat mixing metric
mean(seurat_mixing)
```

299.848132009429

```
In [26]: # Calculate the standard deviation of the Seurat mixing metric
sd(seurat_mixing)
```

3.07534602185969

```
In [27]: # Calculate the mixing metric for Harmony integrated data
harmony_mixing <- MixingMetric(sc_datasets.harmony,
                                'seurat_clusters',
                                reduction = "harmony",
                                dims = 1:2,
                                k = 5,
                                max.k = 300,
                                eps = 0,
                                verbose = TRUE
                                )
```

```
In [28]: # Calculate the mean of the Harmony mixing metric
mean(harmony_mixing)
```

280.939317094078

```
In [29]: # Calculate the standard deviation of the Harmony mixing metric
sd(harmony_mixing)
```

41.6904351694389

Let's compare when using no integration, Seurat, and Harmony:

```
In [ ]: # Identify the top 2000 most variable features (genes) using the "vst" method.
# These features are used for downstream analysis.
sc_datasets <- FindVariableFeatures(sc_datasets, selection.method = "vst", nfeatures = 2000) %>%

# Scale and center the data for the identified variable features.
# This standardizes the expression values across cells.
ScaleData(verbose = FALSE) %>%

# Perform Principal Component Analysis (PCA) to reduce dimensionality.
# Retain the first 30 principal components for downstream analysis.
RunPCA(npcs = 30, verbose = FALSE) %>%

# Apply Uniform Manifold Approximation and Projection (UMAP) for non-linear di
# Use the PCA results as input and the first 30 principal components.
RunUMAP(reduction = "pca", dims = 1:30) %>%

# Compute a shared nearest neighbor (SNN) graph for clustering.
# This step identifies relationships between cells based on their UMAP embeddi
FindNeighbors(reduction = "pca", dims = 1:30) %>%

# Perform clustering of cells using the SNN graph.
# Resolution controls the granularity of the clusters (higher resolution = mor
FindClusters(resolution = 0.5)
```

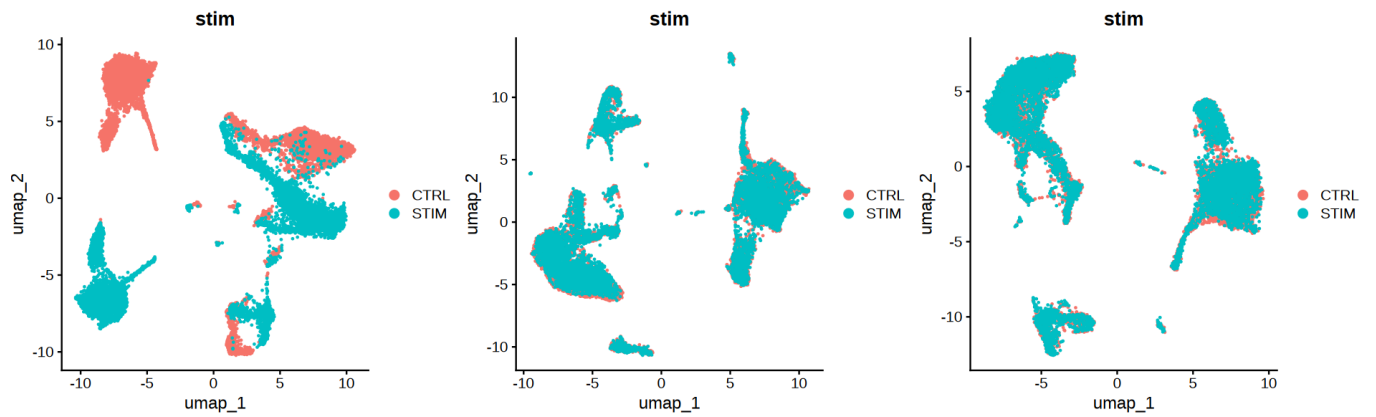
```
In [31]: # Set the size of the output visualization.
options(repr.plot.height = 5, repr.plot.width = 16)

# Create a UMAP plot grouping cells by the "stim" metadata column.
p1 <- DimPlot(sc_datasets, reduction = "umap", group.by = "stim")

# Create a UMAP plot for the integrated dataset, grouping cells by the "stim" metadata column.
p2 <- DimPlot(sc_datasets.combined, reduction = "umap", group.by = "stim")

# Create a UMAP plot for the Harmony-corrected dataset, grouping cells by the "stim" metadata
p3 <- DimPlot(sc_datasets.harmony, reduction = "umap", group.by = "stim")

# Combine the three UMAP plots (p1, p2, and p3) into a single visualization.
p1 + p2 + p3
```



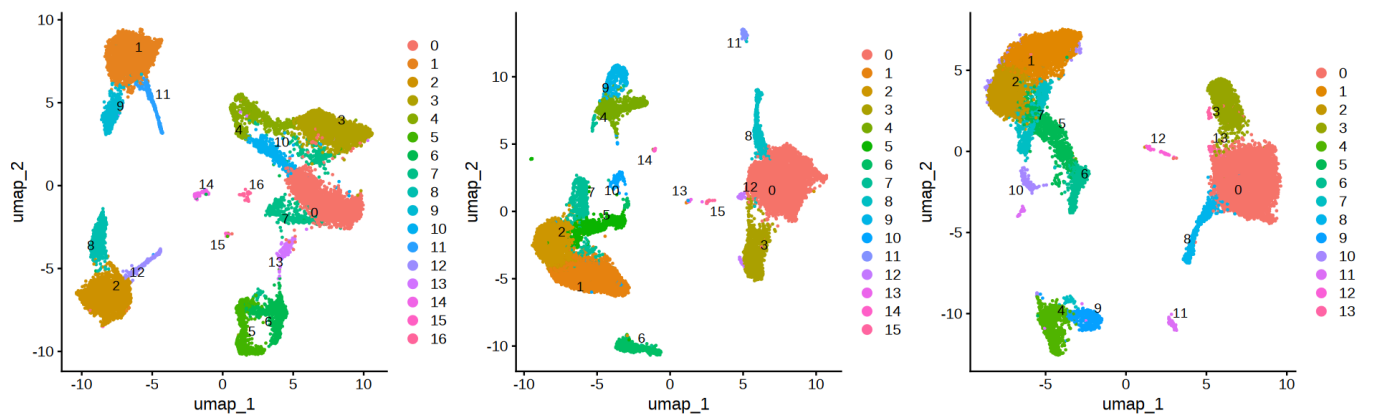
```
In [32]: # Set the height and width of the output visualization.
options(repr.plot.height = 5, repr.plot.width = 16)

# Create a UMAP plot for the original sc_datasets object.
# The 'label' parameter adds labels to the clusters in the plot.
# The 'repel' parameter prevents text labels from overlapping.
p1 <- DimPlot(sc_datasets, reduction = "umap", label = TRUE, repel = TRUE)

# Create a UMAP plot for the combined sc_datasets object.
p2 <- DimPlot(sc_datasets.combined, reduction = "umap", label = TRUE, repel = TRUE)

# Create a UMAP plot for the harmony-corrected sc_datasets object.
# This plot also uses UMAP, with labels and repelling to avoid overlaps.
p3 <- DimPlot(sc_datasets.harmony, reduction = "umap", label = TRUE, repel = TRUE)

p1 + p2 + p3 #Combine plots
```



Extra Questions:

- How different are the results between the Seurat and Harmony integration?
- Why can this happen?
- Can you identify which markers changed in the clusters generated in one method vs the other?

```
In [ ]: ?DimPlot
```

